

Chapitre 6 - ORM - Entity Framework

I.	Objectifs / Intérêt / Enjeu.....	1
II.	Mise en œuvre.....	2
1.	Opérations de base (CRUD) avec Entity Framework.....	2
2.	Clés étrangères et contraintes d'intégrité avec Entity Framework.....	6
III.	Utilisation avec EpokaAbsences	10
1.	Nettoyer le projet servant de base	10
2.	Mettre en place Entity Framework	10
3.	Adapter l'application.....	11
IV.	Conclusion	14
V.	PPE	15

I. Objectifs / Intérêt / Enjeu

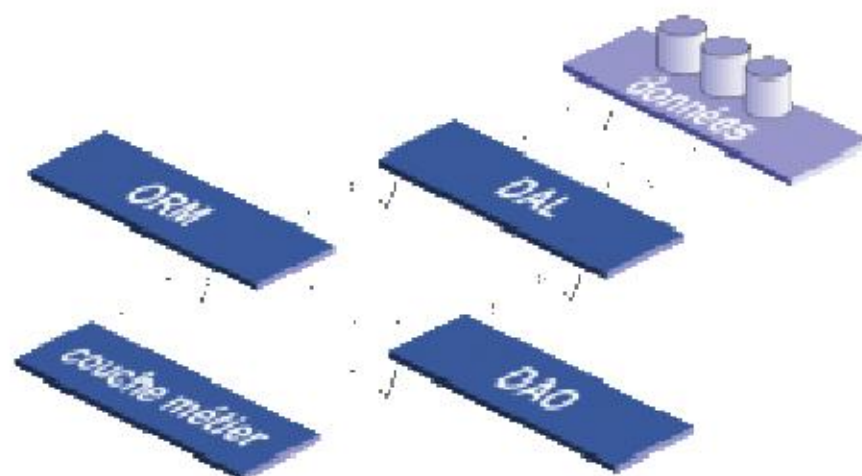
Les objets sont omniprésents dans les langages de programmation modernes. Le problème est leur **persistance**.

Il existe bien quelques bases de données objets, mais celles-ci ne se sont pas imposées face aux SGBD relationnels. Les frameworks ORM permettent donc ce lien, en masquant le travail de conversion d'arrière-plan. Ils réalisent des fonctions assumées habituellement par la couche DAO.

ORM signifie Object-Relational Mapping, c'est-à-dire association objet-relationnel. Le mapping objet-relationnel consiste à associer des objets aux occurrences des bases relationnelles (non objets), en **synchronisant les deux**.

Principaux logiciels ORM :

- Plateforme .NET : *Entity Framework* de Microsoft ;
- Plateforme Java : *Hibernate* ;
- Plateforme PHP 5 : *CakePHP, Laravel, Doctrine* ;
- ...



Rappel :

- DAL = Data Access Layer, couche d'accès aux données (bas niveau). Typiquement, propose les accès « CRUD » sur chaque table. CRUD=Create, Read, Update, Delete ;

- DAO = Data Access Object. Une classe « technique » par table. Propose des méthodes intéressant les classes métiers (recherches multiples, ajout, ...) ;
- Couche métier = Contient des classes avec la logique de l'application. On y retrouve notamment celles associées à la couche DAO.

Avec un ORM, la couche métier ne manipule plus de classes DAO, mais essentiellement des classes métiers !

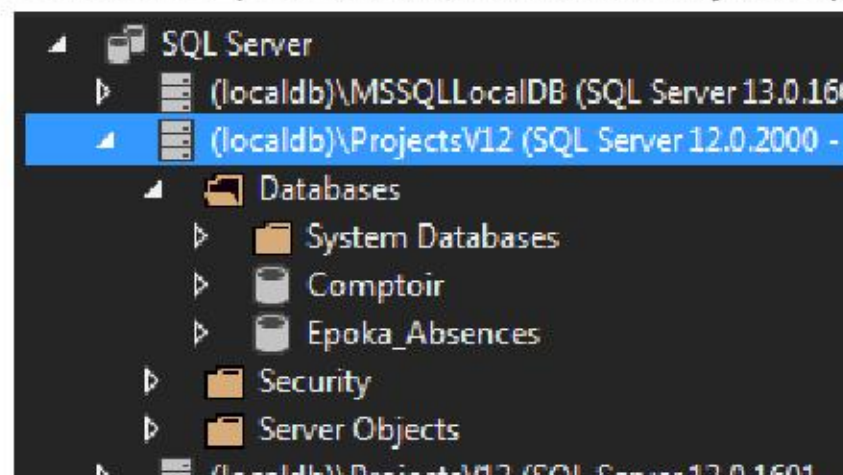
II. Mise en œuvre

Dans Visual Studio 2017, Entity Framework 6 est présent. C'est une version de référence.

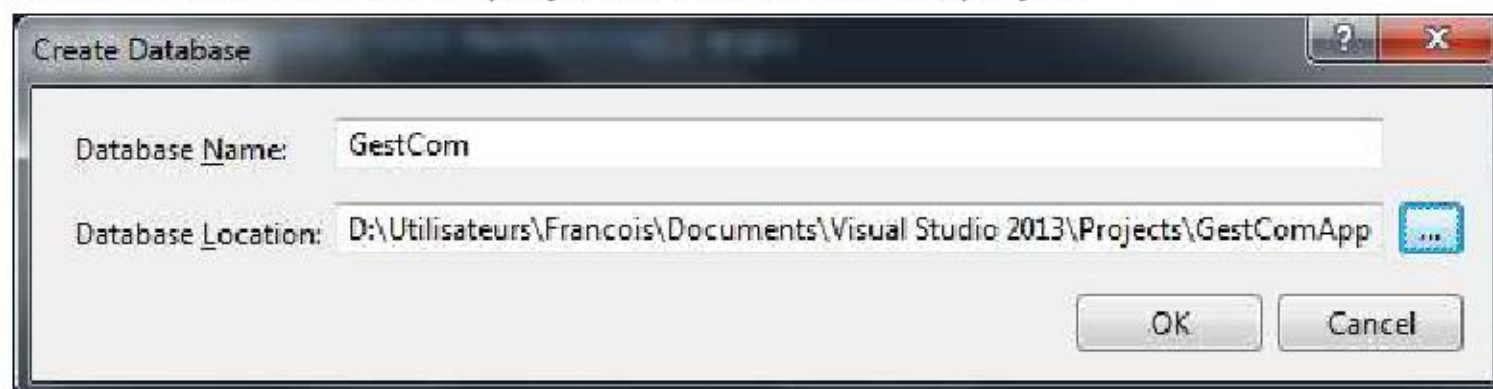
1. Opérations de base (CRUD) avec Entity Framework

a. Création de la base SQL Server : Des articles

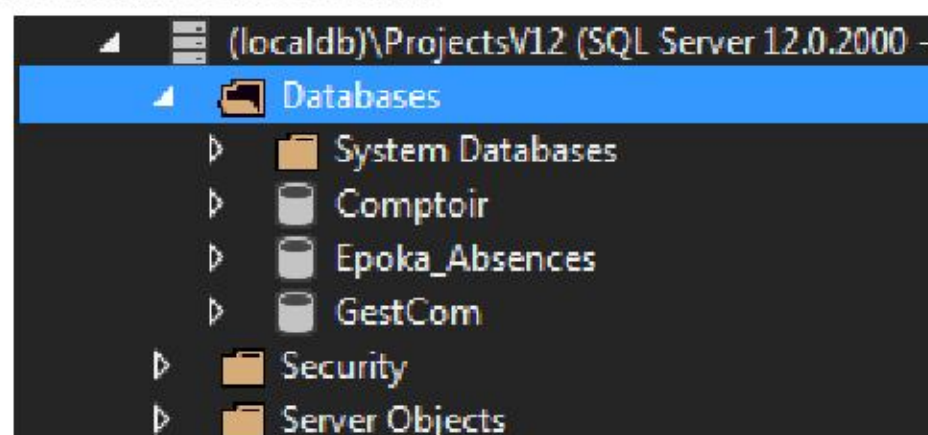
- Créer un projet console nommé *GestComApp* ;
- Afficher l'explorateur d'objets de SQL Server (menu *View, SQL Server Object Explorer*) :



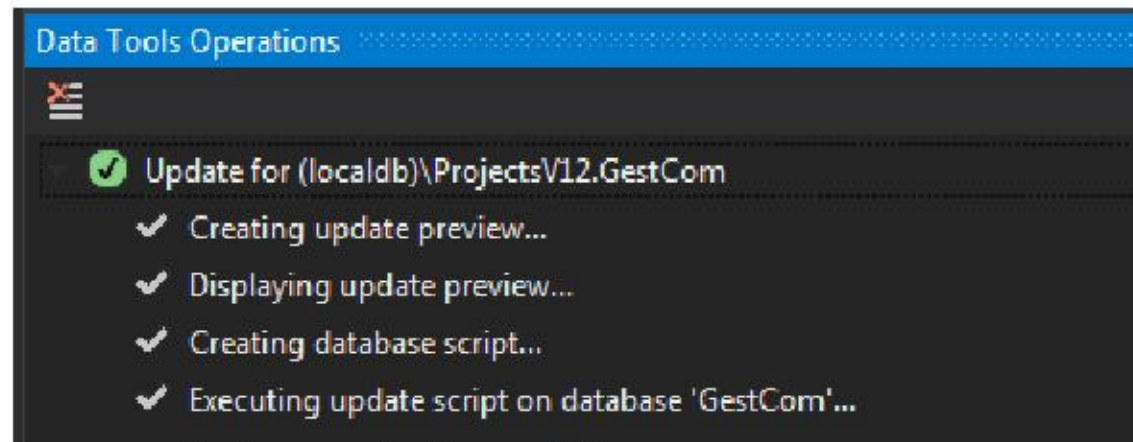
- Plusieurs instances de SQL Server existent. Créer la base *GestCom* (menu contextuel de *Databases*) sur l'instance de *SQL Server version 12*, en la plaçant dans le dossier du projet :



- La base *GestCom* apparaît dans l'instance du serveur :



- Développer la base *GestCom*. Créer une nouvelle table (Menu contextuel de *tables*) ;
- Nommer la table *Article* dans la zone SQL au centre ;
- Créer la colonne *id* : int, not null. Dans *Properties, Identity Specification*, activer (*Is Identity*) => Auto incrément ;
- Créer la colonne *nom* : nvarchar(20), not null ;
- Attention : Pas de majuscule au début des noms de colonnes (comme pour les champs / propriétés) ;
- Cliquer sur *Update* (sous l'onglet), puis *Update Database* ;
- Dans la fenêtre *Data Tools Operations*, vérifier que la coche devient verte ;



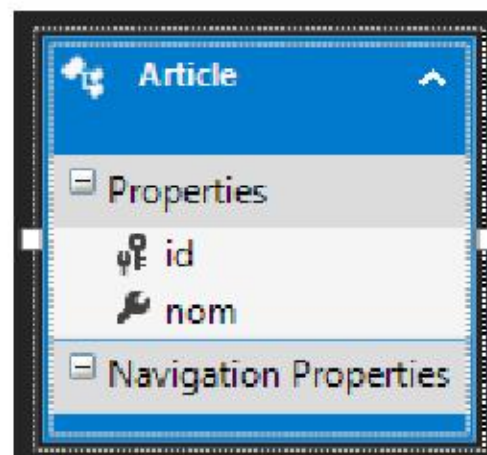
- Fermer l'onglet *dbo.Articles* ;

b. Intégrer Entity Framework au projet

- Dans *Solution Explorer*, menu contextuel sur le projet *GestComApp* : *Add, New Item* ;
- Choisir dans la fenêtre *Visual C# Items, Data, ADO.NET Entity Data Model* ;
- Nommer l'item *GestComModel*, cliquer sur *Add* ;
- Choisir *EF Designer from database* ;



- Cliquer sur *Next* ;
- Pour établir une nouvelle connexion vers SQL Server, choisir *New Connection* ;
- *Data source* doit contenir *Microsoft SQL Server (SqlClient)*. Choisir dans le cas contraire *Change* puis *Microsoft SQL Server*. Valider ;
- Dans *Server name*, indiquer le nom du serveur\nom de l'instance de SQL Server. Observer cette information sur l'explorateur d'objets de SQL Server : *(localdb)\MSSQLLocalDB* ;
- Garder l'authentification *Windows*. Sélectionner la base de données *GestCom* ;
- Tester la connexion puis valider ;
- Cocher l'enregistrement dans *App.Config*, fichier XML ajouté au projet si nécessaire ;
- Dans ce fichier, les données pour se connecter (chaîne de connexion) sont enregistrées sous le nom *GestComEntities* ; Cliquer sur *Next* ;
- Utiliser Entity Framework 6. Cliquer sur *Next* ;
- Cocher *Tables* et développer pour voir la table *Articles* ;
- Cocher *Pluralize and singularize generated object names*. Cela permet que les collections soient au pluriel ;
- Cocher *Include foreign key columns in the model* ;
- Nommer l'espace de nom du modèle *GestComModel*. Cliquer sur *Finish* ;
- Le modèle apparaît :



- Observer le nom de l'objet, *Article*, au singulier, et les propriétés. Fermer le modèle ;
- Dans *Solution Explorer*, le modèle est constitué du fichier *GestComModel.edmx* et de quelques autres, dont *Article.cs*. L'ouvrir :

```
public partial class Article
{
    public int id { get; set; }
    public string nom { get; set; }
}
```

- On retrouve une classe correspondant à la table ; Fermer cette fenêtre ;

c. Ajouter des données

Dans la méthode *main()* de *program*, implémenter le code qui suit :

- Toute opération avec *Entity Framework* débute par l'acquisition du contexte nécessaire :

```
GestComEntities context = new GestComEntities();
```

- Ajouter un article :

```
Article unArticle = new Article
{
    nom = "Ours"
};
```

- L'ajouter à la collection des articles existants :

```
context.Articles.Add(unArticle);
```

- Enregistrer les changements :

```
context.SaveChanges();
```

- Avant d'exécuter, afficher les données de la table *Articles* (menu contextuel sur *dbo.Articles* dans *SQL Server Object Explorer* :

	id	nom
*	NULL	NULL

- Exécuter le code et actualiser les données affichées :

	id	nom
▶	1	Ours
*	NULL	NULL

d. Lire des données - Lister

- Utiliser le contexte d'*Entity Framework* :

```
GestComEntities context = new GestComEntities();
```

- Parcourir les articles :

```
foreach (Article unArticle in context.Articles)
{
    Console.WriteLine("Article : " + unArticle.nom);
}
```

- Attendre de taper <Entrée> pour voir le résultat :

```
Console.ReadLine();
```

e. Lire des données - Rechercher

- Utiliser le contexte d'*Entity Framework* :

```
GestComEntities context = new GestComEntities();
```

- Créer un objet *Article*, qui contienne le premier article dont le nom est *Ourson*. Observer la manière d'indiquer le critère :

```
Article unArticle = context.Articles.FirstOrDefault (a => a.nom == "Ourson");
```

- S'il n'est pas trouvé, afficher *Non trouvé*, sinon, afficher l'id de l'article :

```

if (unArticle == null)
{
    Console.WriteLine("Non trouvé");
}
else
{
    Console.WriteLine(unArticle.id);
}

```

- Attendre de taper <Entrée> pour voir le résultat :
`Console.ReadLine();`
- Exécution : En cherchant *Ourson*, puis en cherchant *Ours* :
Non trouvé 1

f. Modifier des données

- Utiliser le contexte d'*Entity Framework* :
`GestComEntities context = new GestComEntities();`
- Créer un objet *Article*, qui contienne la premier article dont le nom est *Ours* :
`Article unArticle = context.Articles.FirstOrDefault (a => a.nom == "Ours");`
- S'il n'est pas trouvé, afficher *Non trouvé*, sinon,
 - Modifier le nom en *Ourson* ;
 - Enregistrer les changements ;
 - Afficher *Modifié* :

```

if (unArticle == null)
{
    Console.WriteLine("Non trouvé");
}
else
{
    unArticle.nom="Ourson";
    context.SaveChanges();
    Console.WriteLine("Modifié");
}

```

- Attendre de taper <Entrée> pour voir le résultat :
`Console.ReadLine();`
- Avant d'exécuter, afficher les données de la table *Articles* :

	id	nom
▶	1	Ours
*	NULL	NULL

- Exécuter le code et actualiser les données affichées :

	id	nom
▶	1	Ourson
*	NULL	NULL

- L'application indique :
Modifié
- Une deuxième exécution donne :
Non trouvé

g. Supprimer des données

- Utiliser le contexte d'*Entity Framework* :
`GestComEntities context = new GestComEntities();`

- Créer un objet *Article*, qui contienne la premier article dont le nom est *Ourson* :

```
Article unArticle = context.Articles.FirstOrDefault (a => a.nom == "Ourson");
```

- S'il n'est pas trouvé, afficher *Non trouvé*, sinon,
 - Supprimer cet article de la collection *Articles* (donc de la table) ;
 - Enregistrer les changements ;
 - Afficher *Supprimé* :

```
if (unArticle == null)
{
    Console.WriteLine("Non trouvé");
}
else
{
    context.Articles.Remove (unArticle);
    context.SaveChanges();
    Console.WriteLine("Supprimé");
}
```

- Attendre de taper <Entrée> pour voir le résultat :

```
Console.ReadLine();
```

- Avant d'exécuter, afficher les données de la table *Articles* :

	id	nom
▶	1	Ourson
*	NULL	NULL

- Exécuter le code et actualiser les données affichées :

	id	nom
*	NULL	NULL

- L'application indique :

Supprimé

- Une deuxième exécution donne :

Non trouvé

2. Clés étrangères et contraintes d'intégrité avec Entity Framework

a. Modification de la base SQL Server : Des catégories d'articles

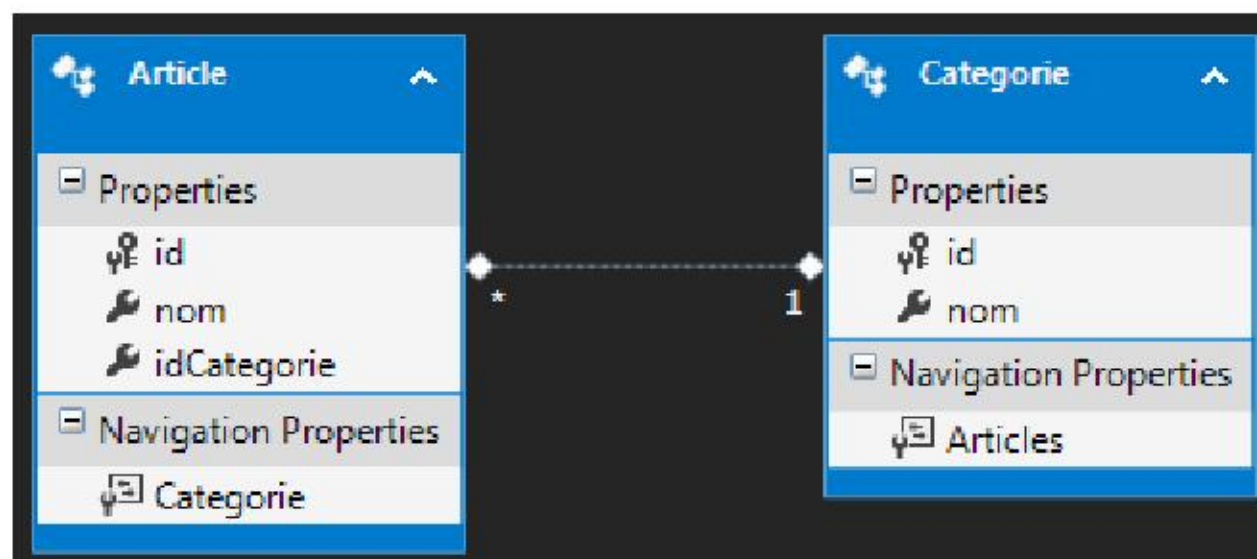
- Via l'*explorateur d'objets de SQL Server*, créer une nouvelle table dans *GestCom* (Menu contextuel de *tables*) ;
- Nommer la table *Categorie* dans la zone SQL au centre ;
- Créer la colonne *id* : int, not null. Dans *Properties*, *Identity Specification*, activer (*Is Identity*) => Auto incrément ;
- Créer la colonne *nom* : nvarchar(20), not null ;
- Attention encore aux minuscules ;
- Cliquer sur *Update* (sous l'onglet), puis *Update Database* ;
- Dans la fenêtre *Data Tools Operations*, vérifier que la coche devient verte :
 **Update for (localdb)\ProjectsV12.GestCom**
- Fermer l'onglet *dbo.Categorie* ;
- Modifier la table *Article* (double-cliquer sur la table dans l'*explorateur d'objets de SQL Server*) ;
- Ajouter une colonne *idCategorie* : int, avec le null possible ;
- Ajouter une clé étrangère (à droite, menu contextuel de *Foreign Keys*) ;
- Laisser le nom par défaut de la contrainte et modifier la définition de la contrainte :

```
CREATE TABLE [dbo].[Article] (
    [id] INT IDENTITY (1, 1) NOT NULL,
    [nom] NVARCHAR (20) NOT NULL,
    [idCategorie] INT NOT NULL,
    PRIMARY KEY CLUSTERED ([id] ASC),
    CONSTRAINT [FK_Article_Categorie] FOREIGN KEY ([idCategorie]) REFERENCES [Categorie]([id])
);
```

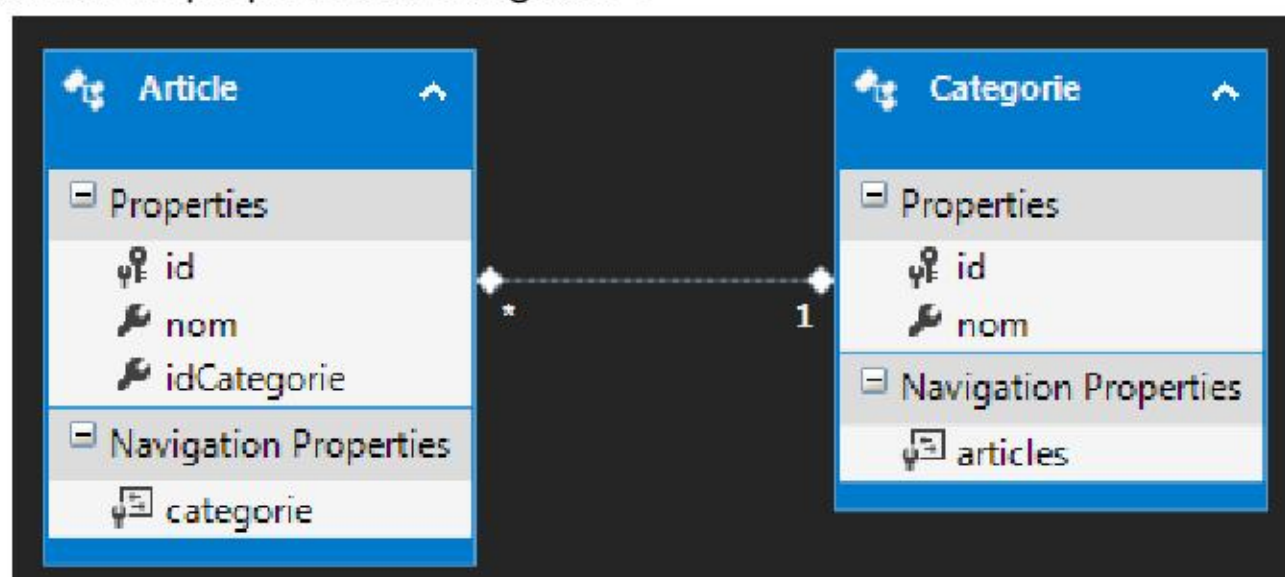
- Cliquer sur *Update* (sous l'onglet), puis *Update Database* ;
- Dans la fenêtre *Data Tools Operations*, vérifier que la coche devient verte :
 **Update for (localdb)\ProjectsV12.GestCom**
- Noter que ni *DROP TABLE* ni *ALTER TABLE* n'ont été requis ;
- Fermer l'onglet *dbo.Article* ;

b. Modifier le modèle de classe d'Entity Framework

- Afficher le modèle en cliquant dans la solution sur *GestComModel.edmx* ;
- Sur le fond, cliquer droit et choisir *Update Model from Database* ;
- Cocher *Tables*. Seule la nouvelle table est présente ;
- Garder coché *Pluralize and singularize generated object names* et *Include foreign key columns in the model* ;
- Cliquer sur *Finish* :



- Observer les propriétés de navigation au singulier (une catégorie dans l'article) ou au pluriel (une collection d'articles par catégorie) ;
- Enlever les majuscules sur les propriétés de navigation :



- Enregistrer le modèle. Observer que les fichiers sous-jacents dans la solution sont mis à jour ;
- Fermer la fenêtre du modèle ;

c. Ajouter des données – Table mère

Dans la méthode *main()* de *program*, implémenter le code qui suit :

- Utiliser le contexte d'Entity Framework :

```
GestComEntities context = new GestComEntities();
```
- Créer deux objets *Categorie*, et les ajouter immédiatement à la collection de la table :

```
context.Categories.Add(new Categorie { nom = "Viande" });
context.Categories.Add(new Categorie { nom = "Poisson" });
```

- Enregistrer les changements :

```
context.SaveChanges();
```

- Avant d'exécuter, afficher les données de la table *Categorie* :

	id	nom
*	NULL	NULL

- Exécuter le code et actualiser les données affichées :

	id	nom
▶	1	Viande
	2	Poisson
*	NULL	NULL

d. Ajouter des données – Table fille – Version 1

On veut ajouter le *Steack haché* à la catégorie *Viande*, en valorisant la clé étrangère (le test de présence de la catégorie *Viande* n'est pas présent) :

- Utiliser le contexte d'*Entity Framework* :

```
GestComEntities context = new GestComEntities();
```

- Rechercher la catégorie *Viande* :

```
Categorie uneCategorie = context.Categories.FirstOrDefault(c => c.nom == "Viande");
```

- Créer un *Article* avec ses propriétés (hors numéro auto-incrémenté) :

```
Article unArticle = new Article { nom="Steack haché", idCategorie=uneCategorie.id };
```

- L'ajouter à la collection des articles :

```
context.Articles.Add(unArticle);
```

- Enregistrer les changements :

```
context.SaveChanges();
```

- Avant d'exécuter, afficher les données de la table *Article* :

	id	nom	idCategorie
*	NULL	NULL	NULL

- Exécuter le code et actualiser les données affichées :

	id	nom	idCategorie
▶	2	Steak haché	1
*	NULL	NULL	NULL

e. Ajouter des données – Table fille – Version 2

On veut ajouter le *Lieu* à la catégorie *Poisson*, en valorisant la propriété de navigation côté *Article* (le test de présence de la catégorie *Poisson* n'est pas présent) :

- Utiliser le contexte d'*Entity Framework* :

```
GestComEntities context = new GestComEntities();
```

- Rechercher la catégorie *Poisson* :

```
Categorie uneCategorie = context.Categories.FirstOrDefault(c => c.nom == "Poisson");
```

- Créer un *Article* avec sa propriété nom et sa propriété de navigation (sans numéro auto-incrémenté) :

```
Article unArticle = new Article { nom = "Lieu", categorie = uneCategorie };
```

- L'ajouter à la collection des articles :

```
context.Articles.Add(unArticle);
```

- Enregistrer les changements :

```
context.SaveChanges();
```

- Avant d'exécuter, afficher les données de la table *Article* :

	id	nom	idCategorie
▶	2	Steak haché	1
*	NULL	NULL	NULL

- Exécuter le code et actualiser les données affichées :

	id	nom	idCategorie
▶	2	Steak haché	1
	3	Lieu	2
*	NULL	NULL	NULL

f. Ajouter des données – Table fille – Version 3

On veut ajouter le *Colin* à la catégorie *Poisson*, en valorisant la propriété de navigation côté *Categorie* (le test de présence de la catégorie *Poisson* n'est pas présent) :

- Utiliser le contexte d'*Entity Framework* :

```
GestComEntities context = new GestComEntities();
```

- Rechercher la catégorie *Poisson* :

```
Categorie uneCategorie = context.Categories.FirstOrDefault(c => c.nom == "Poisson");
```

- Créer un *Article* avec uniquement sa propriété nom :

```
Article unArticle = new Article { nom = "Colin" };
```

- L'ajouter à la collection des articles de la catégorie :

```
uneCategorie.articles.Add(unArticle);
```

- Enregistrer les changements :

```
context.SaveChanges();
```

- Avant d'exécuter, afficher les données de la table *Article* :

	id	nom	idCategorie
▶	2	Steak haché	1
	3	Lieu	2
*	NULL	NULL	NULL

- Exécuter le code et actualiser les données affichées :

	id	nom	idCategorie
▶	2	Steak haché	1
	3	Lieu	2
	4	Colin	2
*	NULL	NULL	NULL

g. Lire des données d'une table à l'autre

On va rechercher la catégorie *Poisson*, puis lire tous les articles de cette catégorie :

- Utiliser le contexte d'*Entity Framework* :

```
GestComEntities context = new GestComEntities();
```

- Rechercher la catégorie *Poisson* :

```
Categorie uneCategorie = context.Categories.FirstOrDefault(c => c.nom == "Poisson");
```

- Pour chaque article de la catégorie, on affiche le nom de l'article :

```
foreach (Article unArticle in uneCategorie.articles)
{
    Console.WriteLine(unArticle.nom);
}
```

- Attendre de taper <Entrée> pour voir le résultat :

```
Console.ReadLine();
```

- Exécution :

```
Lieu
Colin
```

h. Exercice

Afficher le nom de la catégorie de l'article *Steack haché* en suivant le même principe que précédemment.

Correction :

```
GestComEntities context = new GestComEntities();
Article unArticle = context.Articles.FirstOrDefault(a => a.nom == "Steack haché");
Console.WriteLine(unArticle.categorie.nom);
Console.ReadLine();
```

Compléter en affichant le nombre d'articles figurant dans la catégorie indiquée.

Correction :

```
GestComEntities context = new GestComEntities();
Article unArticle = context.Articles.FirstOrDefault(a => a.nom == "Lieu");
Console.WriteLine(unArticle.categorie.nom);
Console.WriteLine(unArticle.categorie.articles.Count);
Console.ReadLine();
```

III. Utilisation avec EpokaAbsences

Pour mettre en place la version ORM, les étapes classes DAO et procédures stockées sont sans impact puisque l'accès aux tables est totalement pris en charge par l'ORM ...

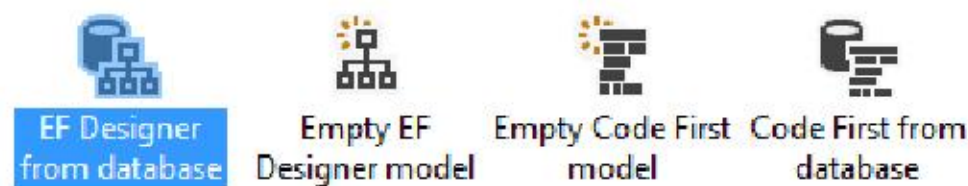
1. Nettoyer le projet servant de base

- Reprendre le projet du chapitre précédent (procédures stockées) ;
- Supprimer de la solution tous les fichiers en rapport avec les classes métiers (générées au sein de Visual Studio via le diagramme de classe), et DAO, soit :
 - Absence.cs ;
 - AbsenceDAO.cs ;
 - AccesSGBD.cs ;
 - ClassDiagramm.cd ;
 - Motif.cs ;
 - MotifDAO.cs ;
 - Salarie.cs ;
 - SalarieDAO.cs ;

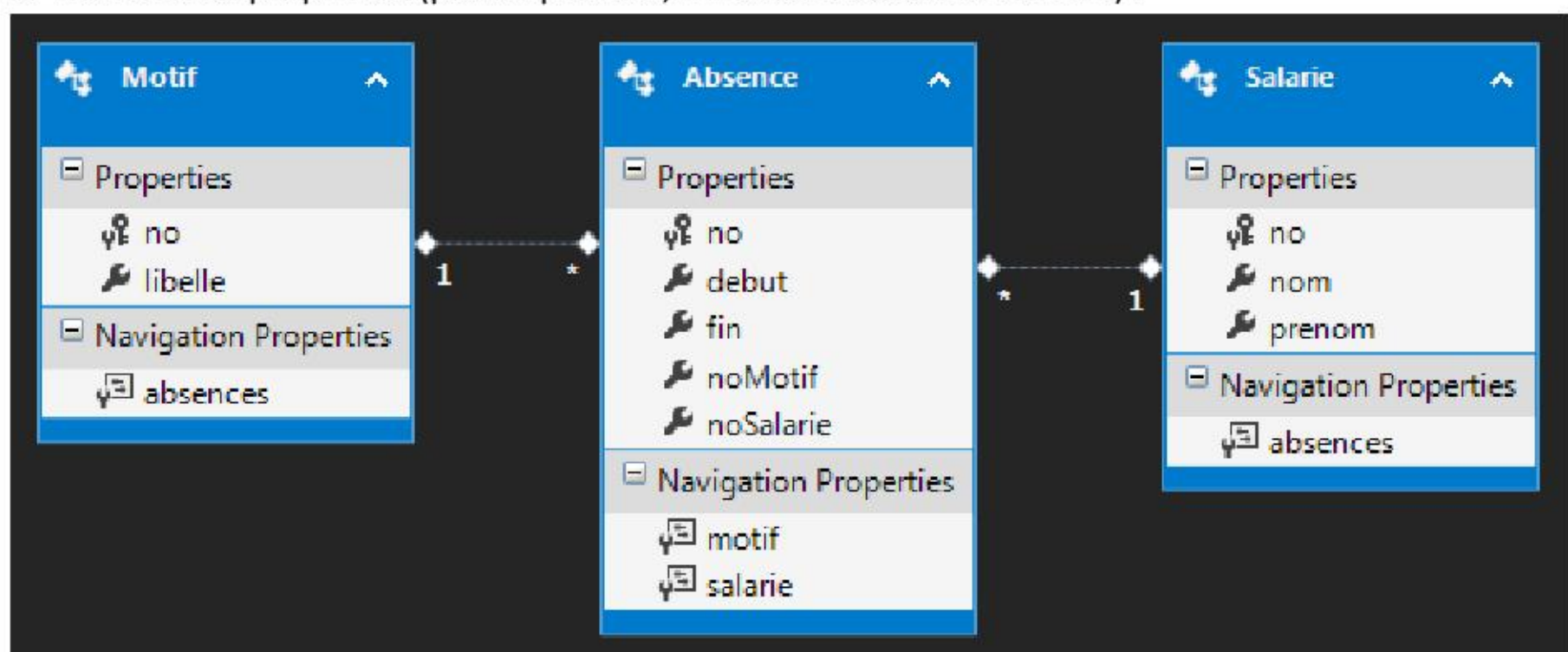
2. Mettre en place Entity Framework

- Ajouter ADO.NET Entity Data Model (depuis le projet, *Add, New Item, Visual C# Items, Data, ADO.NET Entity Data Model* ;
- Nommer l'item *EpokaAbsencesModel*, cliquer sur *Add* ;

- Choisir *EF Designer from database* ;



- Cliquer sur *Next* ;
- Pour établir une nouvelle connexion vers SQL Server, choisir *New Connection* ;
- *Data source* doit contenir *Microsoft SQL Server (SqlClient)*. Choisir dans le cas contraire *Change* puis *Microsoft SQL Server*. Valider ;
- Dans *Server name*, indiquer le nom du serveur\nom de l'instance de SQL Server. Observer cette information sur l'explorateur d'objets de SQL Server : *(localdb)\ProjectsV13* ;
- Garder l'authentification Windows. Sélectionner la base de données *Epoka_Absences* ;
- Tester la connexion puis valider ;
- Cocher l'enregistrement dans *App.Config*, fichier XML ajouté au projet si nécessaire ;
- Dans ce fichier, les données pour se connecter (chaîne de connexion) sont enregistrées sous le nom *EpokaAbsencesEntities* ; Cliquer sur *Next* ;
- Utiliser Entity Framework 6. Cliquer sur *Next* ;
- Cocher *Tables* et développer pour voir les 3 tables ;
- Cocher *Pluralize and singularize generated object names*. Cela permet que les collections soient au pluriel ;
- Cocher *Include foreign key columns in the model* ;
- La base utilisée contient des procédures stockées, qu'il est possible d'importer. Ne pas le faire ;
- Nommer l'espace de nom du modèle *EpokaAbsencesModel*. Cliquer sur *Finish* ;
- Le modèle apparaît ;
- Rectifier les noms de propriétés (pas de préfixes, minuscule au début du nom) :



- Fermer le modèle ;
- Dans *Solution Explorer*, on retrouve les fichiers de classe (sous le modèle *EpokaAbsencesModel.edmx*) ;
- Les observer puis les refermer ;

3. Adapter l'application

Modifier le code dans la fenêtre *FMere*. Débuter avec la méthode *majListeSalaries* :

- Ne changer que le parcours de la collection des Salariés :

```

private void majListeSalaries()
{
    // Effacer la liste de l'onglet Salaries
    lvSalaries.Items.Clear();
    // Effectuer la dé-selection, non produite automatiquement
    lvSalaries_SelectedIndexChanged(null, null);

    // Effacer aussi la liste déroulante des salariés de l'onglet Absences
    cbSalarie.Items.Clear();
    // Effectuer la dé-selection, non produite automatiquement
    cbSalarie_SelectedIndexChanged(null, null);

    EpokaAbsencesEntities context = new EpokaAbsencesEntities();
    foreach (Salarie unSalarie in context.Salaries) {
        // Remplissage de la liste de l'onglet Salaries
        ListViewItem lvi = new ListViewItem();
        lvi.Text = unSalarie.no.ToString();
        lvi.SubItems.Add(unSalarie.nom);
        lvi.SubItems.Add(unSalarie.prenom);
        lvSalaries.Items.Add(lvi);

        // Remplissage de la liste déroulante de l'onglet Absences
        ComboBoxItem cbi = new ComboBoxItem();
        cbi.code = unSalarie.no.ToString();
        cbi.libelle = unSalarie.nom + " " + unSalarie.prenom;
        cbSalarie.Items.Add(cbi);
    }
}

```

Pour la méthode *b_AjouterSalarie_Click*, très peu de différence :

```

private void bAjouterSalarie_Click(object sender, EventArgs e)
{
    EpokaAbsencesEntities context = new EpokaAbsencesEntities();
    Salarie unSalarie = new Salarie();
    unSalarie.nom = tbNom.Text;
    unSalarie.prenom = tbPrenom.Text;
    context.Salaries.Add(unSalarie);
    context.SaveChanges();
    majListeSalaries();
}

```

Méthode *b_ModifierSalarie_Click* :

```

private void bModifierSalarie_Click(object sender, EventArgs e)
{
    EpokaAbsencesEntities context = new EpokaAbsencesEntities();
    int noSalarie = Int32.Parse(lvSalaries.SelectedItem[0].Text);
    Salarie unSalarie = context.Salaries.FirstOrDefault
        (s => s.no == noSalarie);
    if (unSalarie == null)
    {
        MessageBox.Show("Le salarié n'existe plus");
    }
    else
    {
        unSalarie.nom = tbNom.Text;
        unSalarie.prenom = tbPrenom.Text;
        context.SaveChanges();
    }
    majListeSalaries();
}

```

Méthode *B_SupprimerSalarie_Click* :

```

private void bSupprimerSalarie_Click(object sender, EventArgs e)
{
    EpokaAbsencesEntities context = new EpokaAbsencesEntities();
    int noSalarie = Int32.Parse(lvSalaries.SelectedItem[0].Text);
    Salarie unSalarie = context.Salaries.FirstOrDefault
        (s => s.no == noSalarie);
    if (unSalarie == null)
    {
        MessageBox.Show("Le salarié n'existe plus");
    }
    else
    {
        context.Salaries.Remove (unSalarie);
        context.SaveChanges();
    }
    majListeSalaries();
}

```

Reste la méthode *majListeAbsences*. Elle nécessite comme auparavant :

- Un parcours des motifs, pour remplir les groupes du ListView ;
- Un parcours des absences du salarié ;

```

private void majListeAbsences()
{
    lvAbsences.Items.Clear(); // Effacer la liste de l'onglet Absences

    // Remplir les ListViewGroup avec les motifs
    EpokaAbsencesEntities context = new EpokaAbsencesEntities();
    foreach (Motif unMotif in context.Motifs)
    {
        ListViewGroup lvg = new ListViewGroup(unMotif.libelle);
        lvAbsences.Groups.Add(lvg);
    }

    if (cbSalarie.SelectedItem != null)
    {
        // Remplir les ListViewItem avec les absences du salarié,
        // en les rattachant au bon ListViewGroup (motif)
        int noSalarie=Int32.Parse((cbSalarie.SelectedItem as ComboBoxItem).code.ToString());
        Salarie unSalarie = context.Salaries.FirstOrDefault(s => s.no == noSalarie);
        if (unSalarie == null)
        {
            MessageBox.Show("Le salarié n'existe plus");
        }
        else
        {
            foreach (Absence uneAbsence in unSalarie.absences)
            {
                ListViewItem lvi = new ListViewItem();
                lvi.Text = uneAbsence.debut.ToString("ddd dd/MM/yyyy");
                lvi.SubItems.Add(uneAbsence.fin.ToString("ddd dd/MM/yyyy"));
                TimeSpan ts = uneAbsence.fin - uneAbsence.debut;
                lvi.SubItems.Add((ts.Days + 1).ToString() + " jour(s)");

                // Rechercher le groupe :
                foreach (ListViewGroup lvg in lvAbsences.Groups)
                {
                    if (lvg.Header.ToString() == uneAbsence.motif.libelle)
                    {
                        lvi.Group = lvg;
                    }
                };
                lvAbsences.Items.Add(lvi);
            }
        }
    }
}

```

IV. Conclusion

Réalisé avec beaucoup moins de travail. Les avantages :

- De la sécurité dans les requêtes SQL, (requêtes maîtrisées, pas d'injection SQL possible, ...) ;
- Adapté aux gros projets ;
- La maintenance / évolution est facilitée ;
- Sécurité dans le code (erreur SQL interceptée) ;
- La migration vers un autre support de stockage (uniquement SGBD, ...) est aisée, si le pilote utilisé par *Entity Framework* existe ;

V. PPE

Vous êtes chargé de réaliser la phase 2 en recourant à l'approche ORM.

Prenez le projet de ce chapitre, et complétez-le avec le code déjà réalisé du chapitre précédent, que vous adapterez.

Gardez bien des projets distincts pour chaque chapitre.